VAAL UNIVERSITY OF TECHNOLOGY

Inspiring thought. Shaping talent.

FACULTY: Applied and Computer Sciences

DEPARTMENT: Computer Sciences

SUBJECT: Business Analysis 3.2

SUBJECT CODE: AIBUY3A

GROUP CODE: Group name (AI Budget Assistant)

LECTURER: Mr. MA Matsela

MODERATOR: Mrs. MA Matyila

YEAR: 2026

ASSESSMENT NAME:	Group name – AI Budget Assistant
ASSESSMENT DATE:	02/11/2026

STUDENT NUMBER	INITIALS AND SURNAME	SIGNATURE
225092816	L Manyuwa	L
240715748	PM Ndlovu	P
225011468	AA Singo	A
224251473	E Shiringane	E
224639501	MR Mathye	M
223005673	NM Shibambu	N
240567745	NR Modise	N

STUDENT NUMBER	INITIALS AND SURNAME	SIGNATURE

Table of Contents

Declaration **3**

Documentation Aspects **4**

AI Solution (5 Marks) 4

Business Objectives (25 Marks) 4

Business objectives ..... 4 Business success criteria ..... 4 Business background 5

Requirements ..... 5 Constraints ..... 5 Risks 6

Tools and techniques 6

Problem Definition (10 Marks) 6

What is the problem? 6

How relevant is it to the theme, and how beneficial will it be in solving the problem? 7

Poster 8

Theoretical Aspect **9**

Machine Learning Approach (5 Marks) ..... 9 Data (5 Marks) ..... 9 Model Evaluation ..... 10 Time Series Analysis on Data 13

Solution Techniques (5 Marks) 14

Natural Language Processing, Speech Recognition or Speech Synthesis (5 Marks) 15

Deep Learning (5 Marks) 15

Other Features (5 Marks) 16

AI Solution – Practical Aspect (10 Marks) **16**

Train Model Codes 16

Test Model Codes 23

Predict Codes 27

Grammarly Report ..... 29 **References** **29**

Declaration

Declaration of Originality

We, the undersigned, declare that:

- AIown work.
- All sources of information and references used have been acknowledged appropriately.
- No part of this work has been copied without proper citation, nor has it been submitted for assessment in any other course/module.
- We understand that plagiarism is a serious academic offense and confirm that this submission complies with

STUDENT NO	INITIALS AND SURNAME	SIGNATURE
225092816	L Manyuwa	ML NYUWA
240715748	PM Ndlovu	PNDLOVU
225011468	AA SINGO	A SINGO
224251473	E SHIRINGANE	SHIRINGANE E
224639501	MR MATHYE	R MATHYE
223005673	NM SHIBAMBU	N SHIBAMBU
240567745	NR MODISE	N MODISE

Documentation Aspects

AI Solution (5 Marks)

AI Budget Assistant is an AI-based personal finance and budgeting system. A supervised machine learning model (a Random Forest classifier inside a Scikit-learn pipeline) reads the merchant description, amount and date of each

bank transaction and automatically assigns it to one of 18 spending categories such as Groceries, Restaurants, Gas & Fuel and Utilities. The assistant then tracks cumulative spending per category against a s the user at 80% of a category budget and alerts the user at 100%.

The solution is delivered as a command-line assistant (interactive and batch modes) and a Streamlit web interface. By demonstrating how artificial intelligence can transform the financial services industry, it aligns with -making, improves

issues by automating transaction categorisation, budget generation and overspending alerts. On a hold-out test set of 137 unseen transactions the model reaches an accuracy of 95.62%.

Business Objectives (25 Marks)

Business objectives

The primary objective of the AI Budget Assistant is to develop and apply a machine learning classification model that automatically categorises financial transactions, generates a data-driven monthly budget, and detects overspending early. This reduces manual effort, improves financial awareness, and allows users to take control of their spending.

The goals of the AI Budget Assistant are as follows:

- Create a trustworthy AI system that categorises bank transactions into spending categories based on the merchant description, amount and date.
- Help users track their cumulative spending per category and identify areas where they overspend.
- Generate an automatic monthly budget for every category, derived from historical spending data (average monthly spend plus a 10% buffer).
- Provide early feedback on budget utilisation: a warning at 80% of the budget and an alert at 100%.
- Reduce the time and effort required for manual budgeting and expense tracking.
- Increase financial literacy and awareness among users.

Business success criteria

- -out transactions); mean 5-fold cross-validation accuracy is 95.74%, and weighted precision, recall and F1-score are 95.87%, 95.62% and 95.56%.
- The system achieves significantly shorter categorisation times when compared to manual entry: a prediction takes a few milliseconds per transaction, and a whole CSV of transactions can be categorised in one run.
- Users accept the solution and give it favourable reviews after using the generated budget and alerts.
- The model is scalable across different income levels and spending patterns: it can be retrained on any transaction CSV with one command (train_model.py), and column names are configured in one place (config.py).

- For every transaction, the system predicts the spending category in well under two seconds.
- A monthly budget is generated for every spending category in the data (21 categories), set at 110% of the average monthly spend, and spending is monitored against it with 80% and 100% thresholds.

Business background

Managing personal finances is a challenge for many individuals. People often do not know where their money goes each month because manually categorising transactions and creating budgets is time-consuming and errorprone. Manual tracking of personal expenses causes late detection of overspending, so users overspend, fail to save, and accumulate debt.

The financial services sector requires AI software that makes budgeting easy, quick, and precise to solve this problem. By analysing transaction patterns, categorising spending automatically and alerting users while they are still within their budget, the AI Budget Assistant closes this gap and enables users to take action before financial difficulties arise.

Requirements

- Dataset: personal_transactions.csv 806 bank transactions (Jan 2018 to Sep 2019) with the columns Date, Description, Amount, Transaction Type, Category and Account Name. Only debit (expense) transactions are used.
- Testing.csv the 20% hold-out test set, created automatically by train_model.py; Budget.csv the monthly budget per category, created by make_budget.py.
- Python 3 with Pandas, NumPy, Scikit-learn, Joblib and Streamlit (requirements.txt), plus Matplotlib, Seaborn and Tabulate for charts and console tables.
- Labelled dataset of merchant descriptions and target spending categories.
- Evaluation using accuracy, precision, recall, and F1-score.
- A trained model file (models/budget_model.pkl): one Scikit-learn pipeline that bundles the TF-IDF vectoriser, the StandardScaler and the Random Forest, so no separate scaler file is needed.

Constraints

- Dataset Limitation: the model learns from 806 transactions. After keeping expenses only (688) and dropping three categories with fewer than five examples (Electronics & Software, Food & Dining, Entertainment), 681 transactions across 18 categories remain for modelling.
- Merchant Coverage: the training data contains a limited set of merchant names (for example Amazon, Starbucks and Chevron). Merchants never seen in training, such as local retailers, are classified from weaker signals and may be misclassified until the model is retrained with local data.
- Feature Coverage: only the merchant text (TF-IDF over single words and word pairs, 125 terms), the amount and four date features (day of month, day of week, weekend flag, month-end flag) are used; geographic location is not available.
- Class Imbalance: some categories have few examples (Television 8, Fast Food 16), which makes their scores less reliable (Fast Food F1 0.33 on 3 test rows).
- Update Frequency: to guarantee ongoing relevance, periodic retraining using updated transaction data is required.

- Not a Replacement: the system supports financial decision-making but is not a certified financial advisor.
- Lack of Temporal Data: the classifier treats each transaction independently; spending behaviour over time is only used when the budget is generated (monthly aggregation over 21 months).
- Computational: the saved model is about 1 MB and predicts in milliseconds; training with GridSearchCV (72 parameter combinations, 5-fold) takes roughly a minute on a standard laptop.
- Language Limitation: the classifier expects a merchant string and an amount; free-form sentences are not supported in this version.

Risks

Risk	Impact	Probability	Mitigation Strategy
Misclassification	Medium a transaction is placed in the wrong category	**Medium**	The model exposes class probabilities (mean confidence 0.91 on the test set); top-3 predictions with confidence can be shown and the user can correct a category
Data Bias	Medium small dataset, some merchants and categories underrepresented	**High**	Stratified train/test split (implemented); rare categories dropped; consider class weighting or SMOTE and adding local merchant data
Model Overfitting	Medium poor generalisation (train 98.7% vs test 95.6%)	**Low**	5-fold cross-validation (implemented, 95.7% mean); GridSearchCV over max_depth and minimum samples per split and leaf
Privacy Concerns	High transaction data is sensitive	**Medium**	All processing runs locally with no external services; clear terms of service and user consent forms
User Trust	Medium over-reliance on automated budgets and alerts	**Medium**	Budget rule is transparent (average spend x 1.10); thresholds are configurable; education on AI limitations

Tools and techniques

Programming Environment

- Language: Python 3.x
- IDE: Visual Studio Code
- Version Control: Git & GitHub (for project management)

Core Libraries

- Pandas & NumPy (data handling)
- Scikit-learn (machine learning pipeline, GridSearchCV, metrics)
- Matplotlib & Seaborn (visualisation) and Tabulate (console tables)
- Joblib (model persistence)
- Streamlit (web user interface)

Machine Learning Technique

- Algorithm: Random Forest Classifier (ensemble method) inside a Scikit-learn Pipeline
- Preprocessing: ColumnTransformer TfidfVectorizer (1 2 word n-grams) on the merchant text and StandardScaler on five numeric features (amount, day of month, day of week, weekend flag, month-end flag)

- Hyperparameter tuning: GridSearchCV with 5-fold cross-validation over 72 combinations; best model: 50 trees, no depth limit, sqrt features, min split 5, min leaf 1
- Evaluation: accuracy, precision, recall, F1-score, classification reports, confusion matrices and crossvalidation scores

Problem Definition (10 Marks)

What is the problem?

The lack of financial awareness and the difficulty of manual budgeting is a major problem in personal finance. Many people do not track their spending because it is tedious to categorise every transaction by hand. Manual tracking of personal expenses causes late detection of overspending: by the time the user realises a category is over budget, the money is already spent. As a result, people are unaware of where their money goes, which leads to overspending, insufficient savings, and financial stress.

Conventional budgeting tools require manual input and do not automatically learn from user behaviour. Financial institutions may offer basic categorisation, but these are often rigid and do not generate personalised budgets or early warnings. User outcomes suffer as a result of poor financial planning and delayed intervention.

How relevant is it to the theme, and how beneficial will it be in solving the problem?

Given that it tackles one of the most important and significant fields personal finance and banking this

artificial intelligence to address particular problems and boost productivity across a range of industries.

The AI Budget Assistant illustrates this by integrating a machine learning classification model, a crucial AI technology, into the daily process of expense tracking and budgeting in the financial services industry. It transforms a laborious, traditionally manual process into an automated, data-driven, and scalable one that warns users before they overspend. This is consistent with the 4IR vision of using data and intelligent automation to revolutionise industry practices.

Poster

VAAL UNIVERSITY OF TECHNOLOGY
Inspiring thought. Shaping talent.

AI Budget Assistant

AI-powered personal finance & budgeting system

TAALTECH | BUSINESS ANALYSIS 3.2 (AIBUY3A) | FACULTY OF APPLIED AND COMPUTER SCIENCES

95.6%	95.7%	0.956	18	681
Test accuracy	5-fold CV accuracy	Weighted F1-score	Spending categories	Labelled transactions

THE PROBLEM

- Manual expense tracking is slow, tedious and error-prone.
- Overspending is spotted **too late**, after the money is already gone.
- People rarely know where their money goes each month.

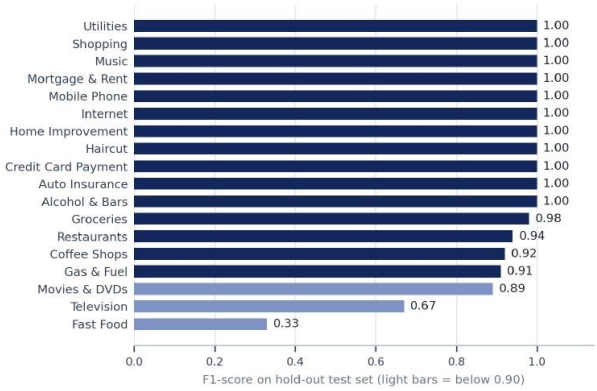
THE AI SOLUTION

- A **Random Forest** reads each transaction (merchant text, amount, date) and assigns a spending category.
- Spending is tracked against an auto-generated monthly budget.
- **Warns at 80%** and **alerts at 100%** of budget.

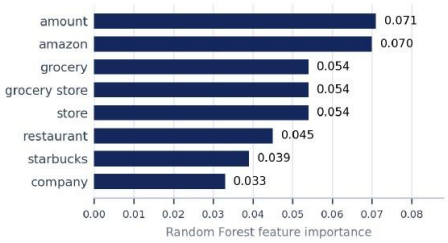
HOW IT WORKS

Transaction (merchant, amount, date) ▶ **Features** (TF-IDF text + scaled numbers) ▶ **Random Forest** (tuned with GridSearchCV) ▶ **Category** (18 spending classes) ▶ **Budget check** (average spend x 1.10) ▶ **Alert** (80% warn / 100% over)

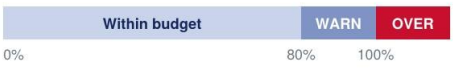
F1-SCORE PER SPENDING CATEGORY (HOLD-OUT TEST SET)



WHAT THE MODEL LEARNS FROM



BUDGET ALERT LOGIC



SUCCESS CRITERIA

- **95.6%** test accuracy; **95.7%** mean 5-fold CV
- Category predicted in milliseconds
- Budget generated for all 21 categories
- Early alerts at 80% and 100% of budget

DATASET & MODEL

- 806 transactions (Jan 2018 - Sep 2019); 681 modelled
- 18 categories; 544 train / 137 test (stratified)
- TF-IDF merchant text + amount + date features
- Random Forest (50 trees), 5-fold GridSearchCV

NEXT: chatbot front-end | natural-language and speech entry | time-series budget forecasting | receipt scanning (CNN)

VAAL UNIVERSITY OF TECHNOLOGY
Inspiring thought. Shaping talent.

GROUP TAALTECH

github.com/TaalTech-Innovators/AI-Budget-Assistant

Theoretical Aspect

Machine Learning Approach

Because the dataset contains transaction features (merchant text, amount and date) and associated spending category labels, supervised machine learning is used in this project. The Random Forest Classifier from Scikitlearn, which combines several decision trees to enhance predictive performance, is used to implement the model. It is wrapped in a Scikit-learn Pipeline together with its preprocessing so that exactly the same transformations are applied during training, testing and prediction.

Random Forest was chosen because it can:

- Manage multi-class classification (18 spending categories).
- Operate efficiently with a large number of sparse features (125 TF-IDF merchant terms plus numeric features).
- Minimise overfitting by averaging predictions from several trees.
- Provide feature importance rankings (which merchant words are most predictive) and class probabilities.

Alternative algorithms were assessed, including single decision trees, k-Nearest Neighbours (KNN), Naive Bayes,

the needs for real-world application, it was chosen.

Data

The transaction dataset (personal_transactions.csv) consists of 806 bank transactions between January 2018 and September 2019. Each record has a date, a merchant description, an amount, a transaction type (debit or credit), an account name and a target label for the spending category. Only debit transactions are treated as expenses, which leaves 688 records in 21 categories.

During pre-processing, amounts are cleaned and converted to numbers, and four date features are derived (day of month, day of week, weekend flag, month-end flag). The merchant text is converted into TF-IDF features and the five numeric features are scaled with StandardScaler. Categories with fewer than five examples are removed so that every class can be stratified, and the remaining 681 transactions are split 80/20 into 544 training and 137 test records (stratified, random_state = 42). The test records are saved to Testing.csv so that evaluation always uses data the model never saw during training.

Dataset fact	Value
Raw transactions	806 (688 debit, 118 credit)
Expense transactions used	688 in 21 categories
Rare categories dropped (fewer than 5 rows)	3 categories, 7 rows (Electronics & Software, Food & Dining, Entertainment)
Rows modelled	681 rows in 18 categories
Train / test split	544 / 137 (80% / 20%, stratified)
Period covered	Jan 2018 to Sep 2019 (21 months)
Largest categories	Groceries 105, Restaurants 81, Credit Card Payment 71, Utilities 63, Shopping 60

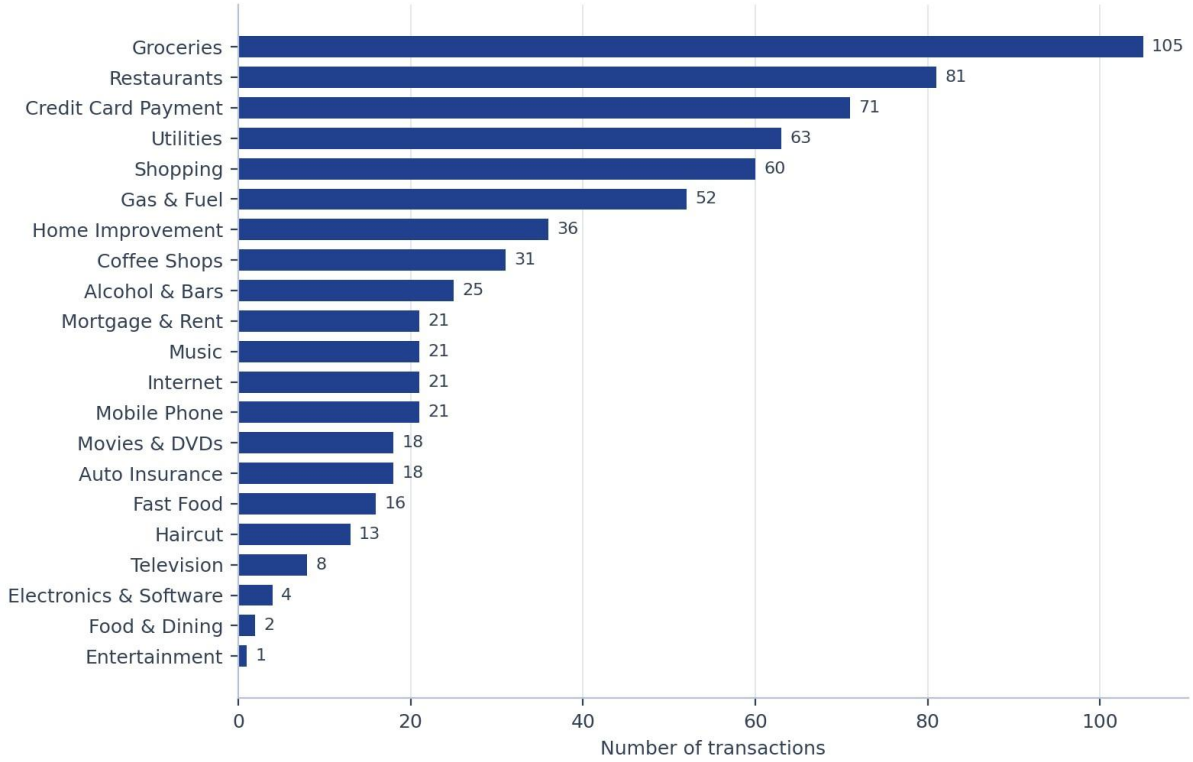


Figure 1: Number of expense transactions per category (688 transactions in 21 categories).

Model Evaluation

The AI model is evaluated on the 137-transaction hold-out test set using standard classification metrics to ensure both reliability and accuracy.

Evaluation Metrics Used

Accuracy	Total percentage of correct category predictions.
Precision	Correctness of positive predictions for each category.
Recall (Sensitivity)	Completeness of predictions (how many actual cases were caught).
F1-score	Harmonic mean of precision and recall for balanced evaluation.
Confusion Matrix	Summarises classification performance across all 18 spending categories.

Results

Metric	Result
Test accuracy	0.9562 (95.62%)
Precision (weighted)	0.9587
Recall (weighted)	0.9562
F1-score (weighted)	0.9556
Training accuracy	0.9871 (overfit gap of 0.031)
5-fold CV accuracy (per fold)	0.9416, 0.9632, 0.9559, 0.9706, 0.9559

5-fold CV accuracy (mean ± std)	0.9574 ± 0.0096
Mean prediction confidence (test set)	0.906

Baseline Comparison

Approach	Accuracy
Random guessing (1 / 18 categories)	5.6%
Most frequent class (always predict Groceries: 21 of 137 test rows)	15.3%
AI Budget Assistant (Random Forest)	95.6%

Per-category results (test set)

Category	Precision	Recall	F1-score	Support
Alcohol & Bars	1.00	1.00	1.00	5
Auto Insurance	1.00	1.00	1.00	4
Coffee Shops	0.86	1.00	0.92	6
Credit Card Payment	1.00	1.00	1.00	14
Fast Food	0.33	0.33	0.33	3
Gas & Fuel	0.91	0.91	0.91	11
Groceries	1.00	0.95	0.98	21
Haircut	1.00	1.00	1.00	3
Home Improvement	1.00	1.00	1.00	7
Internet	1.00	1.00	1.00	4
Mobile Phone	1.00	1.00	1.00	4
Mortgage & Rent	1.00	1.00	1.00	4
Movies & DVDs	0.80	1.00	0.89	4
Music	1.00	1.00	1.00	4
Restaurants	0.94	0.94	0.94	16
Shopping	1.00	1.00	1.00	12
Television	1.00	0.50	0.67	2
Utilities	1.00	1.00	1.00	13

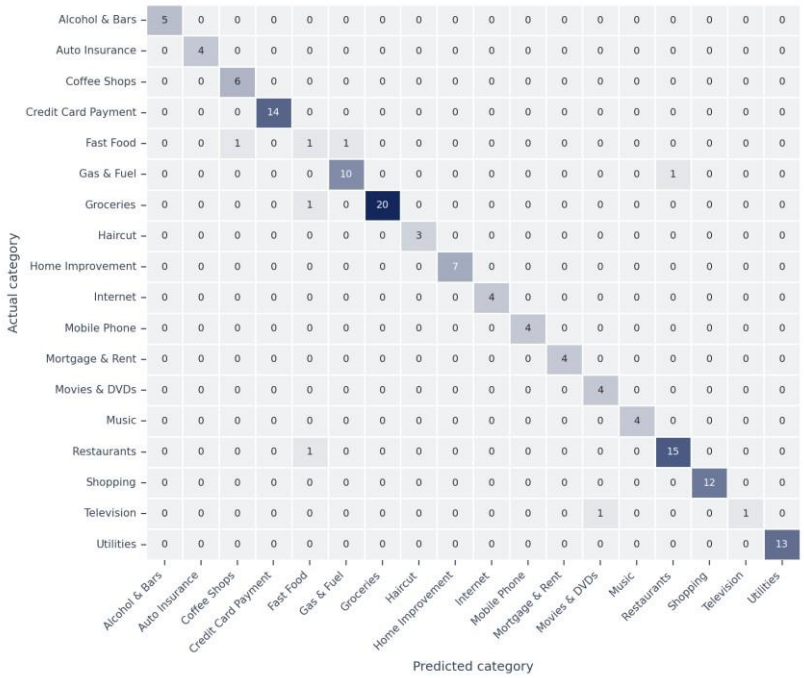


Figure 2: Confusion matrix on the hold-out test set (rows = actual category, columns = predicted category).

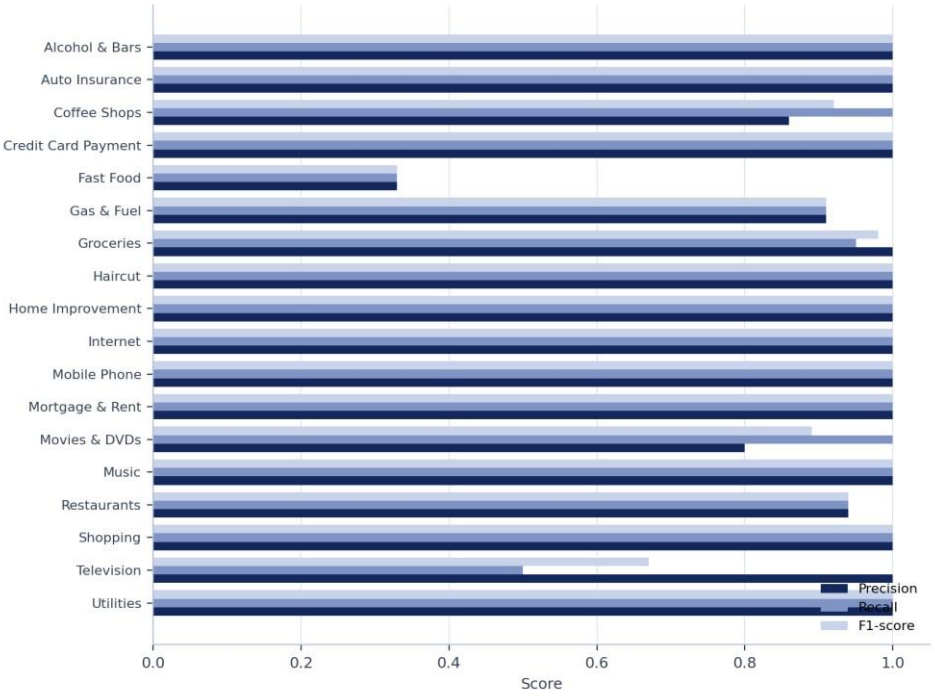


Figure 3: Precision, recall and F1-score per category on the test set.

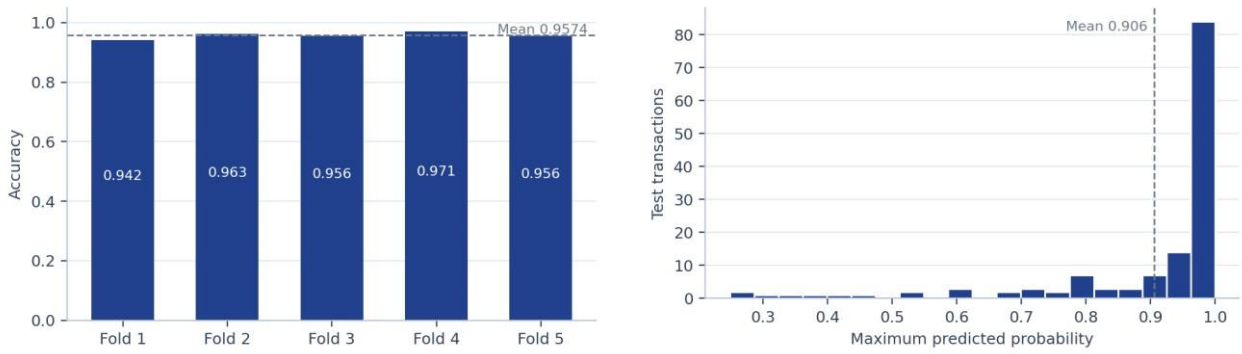


Figure 4: Cross-validation accuracy per fold (left) and distribution of prediction confidence on the test set (right).

Error analysis

Only 6 of the 137 test transactions were misclassified. The errors are concentrated in categories that overlap in meaning or have very few examples:

Merchant	Amount	Actual category	Predicted category
Chick-Fil-A	56.07	Fast Food	Gas & Fuel
Starbucks	9.58	Fast Food	Coffee Shops
Chevron	4.58	Gas & Fuel	Restaurants
Food Truck	24.40	Groceries	Fast Food
Irish Pub	22.00	Restaurants	Fast Food
Netflix	13.90	Television	Movies & DVDs

Time Series Analysis on Data

The classifier itself treats each transaction independently and uses only simple calendar features (day of month, day of week, weekend and month-end flags). However, the system already applies a first form of time-series analysis when it creates the budget: make_budget.py aggregates spending by month for each category over 21 months, computes the average and standard deviation per category, and sets each monthly budget to the average multiplied by 1.10 (a 10% buffer).

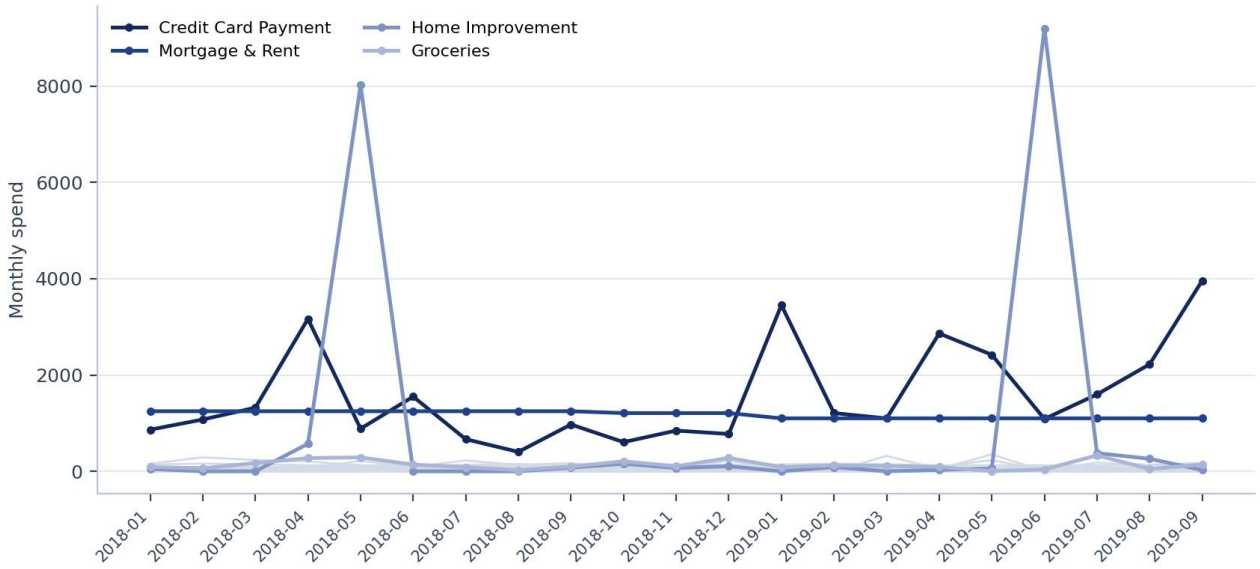


Figure 5: Monthly spend per category, January 2018 to September 2019.

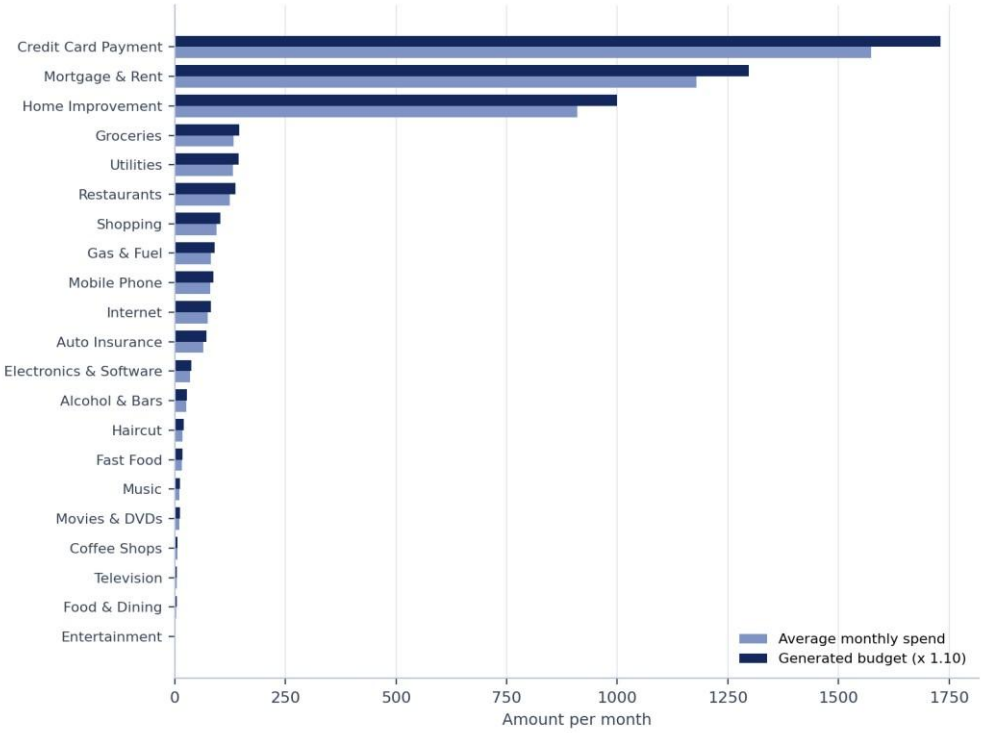


Figure 6: Average monthly spend versus the generated monthly budget for each category.

Generated monthly budget (average monthly spend x 1.10)

Category	Budget	Category	Budget	Category	Budget
Alcohol & Bars	R40	Food & Dining	R43	Mortgage & Rent	R1,297
Auto Insurance	R82	Gas & Fuel	R90	Movies & DVDs	R20
Coffee Shops	R9	Groceries	R146	Music	R12
Credit Card Payment	R1,731	Haircut	R32	Restaurants	R137
Electronics & Software	R264	Home Improvement	R1,500	Shopping	R103

Entertainment	R11	Internet	R82	Television	R14
Category	**Budget**	**Category**	**Budget**	**Category**	**Budget**
Fast Food	R40	Mobile Phone	R88	Utilities	R145

Total monthly budget: R5,886 across 21 categories.

In future versions, time- -month spend instead of using a simple average, detecting seasonal spending patterns (e.g., higher spending in December), and continuous monitoring and proactive budget adjustments.

Solution Techniques

Feature Engineering: TF-IDF on the merchant text (lower-cased, single words and word pairs, 125 terms), plus the cleaned amount and four date features; numeric features are standardised.

Feature Selection: Random Forest feature importances identify the most significant merchant keywords and numeric features (see Figure 7).

Hyperparameter Tuning: GridSearchCV over number of trees (50, 100, 150), tree depth (none, 10, 20), features per split (sqrt, log2), minimum samples to split (2, 5) and per leaf (1, 2) 72 combinations, each scored with 5-fold cross-validation.

Cross-validation: 5-fold CV on the full dataset confirms that results are robust (mean accuracy 95.74%, standard deviation 0.96%).

Data Preparation: only expense transactions are kept, rare categories (fewer than five examples) are removed, and the train/test split is stratified so every category appears in both sets.

Balancing Data: techniques like SMOTE (Synthetic Minority Oversampling) or class weights may be applied for the smallest categories if class imbalance becomes a problem.

These techniques give a test accuracy of 95.62% and a mean cross-validation accuracy of 95.74% while keeping the gap between training accuracy (98.7%) and test accuracy small, which indicates limited overfitting.

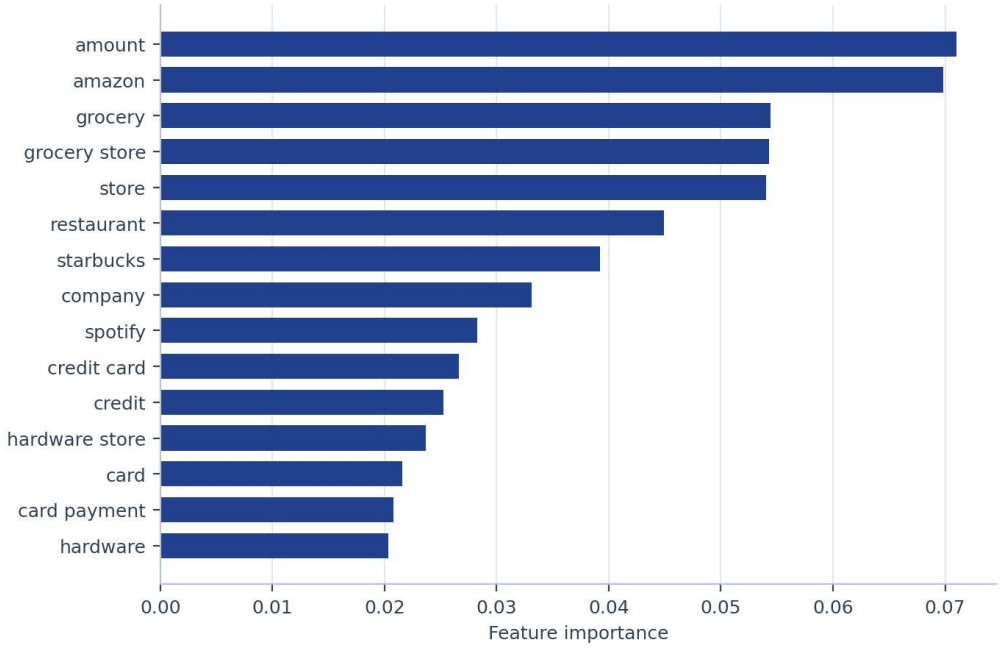


Figure 7: Top 15 feature importances of the trained Random Forest (merchant words and numeric features).

Natural Language Processing, Speech Recognition or Speech Synthesis (5 Marks)

Natural Language Processing is already used in this version: each merchant description is lower-cased and converted by a TF-IDF vectoriser into weighted single-word and two-

In future versions, the AI Budget Assistant can extend this to improve accessibility and user interaction:

Speech recognition could improve usability for users who prefer speaking their transactions instead of typing. This improvement is applicable and doable because Python libraries such as NLTK and SpeechRecognition can be used.

Deep Learning

Although the current model uses Random Forest, Deep Learning techniques could improve performance in advanced phases:

Neural Networks (MLP) or text embeddings for capturing complex interactions between merchant words and amounts, and for recognising merchants not seen in training.

Recurrent Neural Networks (RNN/LSTM) for analysing time-dependent spending data and forecasting budgets.

CNNs could be integrated if visual data (receipt images, bank statements) are added.

Other Features

Besides the trained model, the project includes several features that make the assistant usable and easy to inspect:

Command-line assistant (budget_assistant.py): interactive mode, where the user types a merchant and amount for each transaction, and batch mode for a whole CSV of transactions.

Streamlit web interface (app.py): an editable transaction table, a categorised transactions table, a budget summary with remaining amounts and percentage used, and warning/over-budget alerts.

Alert engine: spending per category is compared with its budget; at 80% the user receives a warning, at 100% an over-budget alert (thresholds are set in config.py).

Automatic reporting: charts (budget versus spent, budget utilisation, spending distribution, daily spend) and a full HTML report (report.py) are saved to the outputs folder.

Batch prediction (predict.py): categorises a CSV file and saves predictions.csv.

The AI Budget Assistant can also incorporate a chatbot as a front-end interface. Users interact with the chatbot to report expenses conversationally. The chatbot queries the AI model to provide spending category predictions and budget advice. It can be available 24 hours, making the solution more engaging and user-friendly. This feature is relevant, well-planned, and appropriate for deployment in banking apps or personal finance websites.

AI Solution Practical Aspect

Train Model Codes

train_model.py loads and cleans the transactions, prints exploratory tables and charts, drops rare categories, creates the 80/20 stratified split (saving Testing.csv), builds the TF-IDF + StandardScaler + Random Forest pipeline, tunes it with GridSearchCV, evaluates it, runs 5-fold cross-validation and saves models/budget_model.pkl. It relies on two small modules that hold the settings and the data preparation. **config.py paths, column names and alert thresholds**

```
import os

# ---------- Paths ----------
BASE_DIR   = os.path.dirname(os.path.abspath(__file__))
DATA_DIR   = os.path.join(BASE_DIR, "data")
MODEL_DIR  = os.path.join(BASE_DIR, "models")
OUTPUT_DIR = os.path.join(BASE_DIR, "outputs")

TRAIN_CSV_PATH  = os.path.join(DATA_DIR, "personal_transactions.csv")
TEST_CSV_PATH   = os.path.join(DATA_DIR, "Testing.csv")
BUDGET_CSV_PATH = os.path.join(DATA_DIR, "budget.csv")
MODEL_PATH      = os.path.join(MODEL_DIR, "budget_model.pkl")
  for d in (MODEL_DIR,
OUTPUT_DIR):     os.makedirs(d,
exist_ok=True)

# ---------- Column names in your real CSV ----------
COL_DATE     = "Date"
COL_MERCHANT = "Description"
COL_AMOUNT   = "Amount"
COL_TYPE     = "Transaction Type"
COL_TARGET   = "Category"
COL_ACCOUNT  = "Account Name"

# Only these count as expenses
EXPENSE_TYPES = ["debit"]

# ---------- Alert thresholds ----------
WARN_THRESHOLD   = 0.80
DANGER_THRESHOLD = 1.00
```

preprocessing.py cleaning and date features

```
import pandas as pd

from config import (
    COL_DATE, COL_MERCHANT, COL_AMOUNT, COL_TYPE, COL_TARGET,     EXPENSE_TYPES,
)
    def load_and_prepare(csv_path, require_target=True):     df =
pd.read_csv(csv_path)
    df = df.loc[:, ~df.columns.str.contains("^Unnamed")]     df.columns = [c.strip() for
c in df.columns]
    # Keep expenses only     if COL_TYPE in df.columns:         df =
df[df[COL_TYPE].astype(str).str.lower().isin(EXPENSE_TYPES)].copy()

    # Clean amounts
    amt = (df[COL_AMOUNT].astype(str)
           .str.replace("R", "", regex=False)
           .str.replace(" ", "", regex=False)            .str.replace(",", "", regex=False))
    df[COL_AMOUNT] = pd.to_numeric(amt, errors="coerce").fillna(0).abs()
    # Date features
    dt = pd.to_datetime(df[COL_DATE], errors="coerce", dayfirst=False)     df["date"]         = dt
    df["day_of_month"]    =    dt.dt.day.fillna(15).astype(int)                  df["day_of_week"]        =
dt.dt.dayofweek.fillna(2).astype(int)
```

```
    df["is_weekend"]   = df["day_of_week"].isin([5,
6]).astype(int)     df["is_month_end"] = (df["day_of_month"] >=
25).astype(int)
    # Standard names used by the model
    df = df.rename(columns={
COL_MERCHANT: "merchant",
        COL_AMOUNT:   "amount",
        COL_TARGET:   "category",
    })       if require_target and "category" not in df.columns:
raise ValueError("Target column 'category' not found in CSV.")


    return df.reset_index(drop=True)
```

train_model.py

```
"""
train_model.py
Trains the AI Budget Assistant categoriser and prints rich tables +
charts. """   import os import warnings
warnings.filterwarnings("ignore")
  import numpy as
np import pandas
as pd import
joblib


import matplotlib
matplotlib.use("Agg")               # safe for all
environments import matplotlib.pyplot as plt import seaborn as
sns


from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV,
cross_val_score from sklearn.feature_extraction.text import TfidfVectorizer from
sklearn.preprocessing import StandardScaler from sklearn.compose import
ColumnTransformer from sklearn.pipeline import Pipeline from sklearn.metrics
import (
    classification_report, accuracy_score, confusion_matrix
)
from config import TRAIN_CSV_PATH, TEST_CSV_PATH, MODEL_PATH, OUTPUT_DIR from
preprocessing import load_and_prepare
  sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 110


# ======================================================================
# Helper: pretty console table
# ====================================================================== from
tabulate import tabulate
  def show_table(title, df,
float_fmt="{:,.2f}"):     print(f"\n===
{title} ===")
    print(tabulate(df, headers="keys", tablefmt="fancy_grid",
                   floatfmt=",.2f", showindex=False))



# ======================================================================
# 1. Load + clean
# ====================================================================== print("="
* 70)
print("AI BUDGET ASSISTANT — MODEL TRAINING") print("="
* 70)
print(f"\nLoading training data from: {TRAIN_CSV_PATH}")


df = load_and_prepare(TRAIN_CSV_PATH)


# ======================================================================
# 2. EDA tables
#
======================================================================
show_table("Dataset Snapshot (first 10 rows)",
df[["merchant", "amount", "day_of_month",
               "day_of_week", "is_weekend", "is_month_end", "category"]].head(10))
```

```
show_table("Numeric Summary",
           df[["amount", "day_of_month", "day_of_week",
               "is_weekend", "is_month_end"]].describe().round(2).reset_index()
             .rename(columns={"index": "stat"}))
# Class distribution
class_counts = df["category"].value_counts().reset_index() class_counts.columns
= ["category", "count"]
class_counts["% of total"] = (class_counts["count"] / class_counts["count"].sum() * 100).round(2)


show_table("Class Distribution", class_counts)


# Average spend per category
avg_spend = (df.groupby("category")["amount"]
               .agg(["mean", "median", "min", "max", "sum", "count"])
               .round(2)
               .reset_index()
               .sort_values("mean",
ascending=False)) show_table("Spend per Category
(R)", avg_spend)
# ======================================================================
# 3. Graph: Class distribution (bar + pie)
#
======================================================================
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
# Bar chart sns.barplot(data=class_counts.sort_values("count",
ascending=False),             x="count", y="category", ax=axes[0],
palette="viridis")
axes[0].set_title("Transactions per Category", fontsize=14, fontweight="bold")
axes[0].set_xlabel("Number of transactions") axes[0].set_ylabel("") for i, v in
enumerate(class_counts.sort_values("count", ascending=False)["count"]):
axes[0].text(v + 0.5, i, str(v), va="center", fontsize=9)
# Pie chart
axes[1].pie(class_counts["count"],
labels=class_counts["category"],
autopct="%1.1f%%",             startangle=140,
            textprops={"fontsize": 9})
axes[1].set_title("Category Share", fontsize=14, fontweight="bold")


plt.tight_layout()
bar_path = os.path.join(OUTPUT_DIR,
"class_distribution.png") plt.savefig(bar_path,
bbox_inches="tight") plt.close()
print(f"\n[Chart saved] {bar_path}")


# ====================================================================== #
4. Graph: Average spend per category
# ====================================================================== fig,
ax = plt.subplots(figsize=(12, 7))
avg_sorted = avg_spend.sort_values("mean", ascending=True)
sns.barplot(data=avg_sorted, x="mean", y="category", palette="magma", ax=ax)
ax.set_title("Average Spend per Category (R)", fontsize=14,
fontweight="bold") ax.set_xlabel("Mean amount (R)") ax.set_ylabel("") for i,
v in enumerate(avg_sorted["mean"]):     ax.text(v + 5, i, f"R{v:,.0f}",
va="center", fontsize=9) plt.tight_layout()
spend_path = os.path.join(OUTPUT_DIR,
"avg_spend_per_category.png") plt.savefig(spend_path,
bbox_inches="tight") plt.close()
print(f"[Chart saved] {spend_path}")


# ======================================================================
# 5. Drop rare categories
# ====================================================================== MIN_SAMPLES
= 5
counts = df["category"].value_counts()
rare = counts[counts < MIN_SAMPLES].index.tolist()
  print("\n" + "=" * 70) if rare:     print(f"Dropping rare categories
(< {MIN_SAMPLES} samples): {rare}")
    df = df[~df["category"].isin(rare)].reset_index(drop=True) else:     print(f"No
categories with fewer than {MIN_SAMPLES} samples — nothing dropped.")

print(f"Kept {len(df)} rows across {df['category'].nunique()} categories.")
```

```
# ======================================================================
# 6. Hold-out split
# ======================================================================
train_df, test_df = train_test_split(
    df, test_size=0.2, random_state=42, stratify=df["category"]
)
test_df.to_csv(TEST_CSV_PATH, index=False)
  split_table = pd.DataFrame({
"split": ["train", "test"],
    "rows":  [len(train_df), len(test_df)],
"%":     [round(len(train_df)/len(df)*100, 1),
round(len(test_df)/len(df)*100, 1)],
})
show_table("Train / Test Split", split_table)
print(f"Testing split saved to: {TEST_CSV_PATH}")
# ======================================================================
# 7. Preprocessing + Model
#
======================================================================
numeric_cols = ["amount", "day_of_month", "day_of_week",
"is_weekend", "is_month_end"]


preprocessor = ColumnTransformer(
transformers=[
        ("merchant_tfidf",
         TfidfVectorizer(ngram_range=(1, 2), lowercase=True),
         "merchant"),
        ("num", StandardScaler(), numeric_cols),
    ]
)


X_train = train_df.drop(columns=["category", "date"]) y_train
= train_df["category"]
X_test  = test_df.drop(columns=["category", "date"]) y_test
= test_df["category"]


# ======================================================================
# 8. GridSearch
# ======================================================================
pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42, n_jobs=-1)),
])
param_grid = {
    "classifier__n_estimators":      [50, 100, 150],
    "classifier__max_depth":         [None, 10, 20],
    "classifier__max_features":      ["sqrt", "log2"],
    "classifier__min_samples_split": [2, 5],
    "classifier__min_samples_leaf":  [1, 2],
}
print("\n" + "=" * 70)
print("Running GridSearchCV (this may take a minute)...") print("="
* 70)


grid_search = GridSearchCV(pipeline, param_grid,
                           cv=5, scoring="accuracy", n_jobs=-1)
grid_search.fit(X_train, y_train)


best_clf = grid_search.best_estimator_


# Pretty print best params params_table
= pd.DataFrame(
    list(grid_search.best_params_.items()),
columns=["parameter", "value"]
) show_table("Best Hyperparameters", params_table)
print(f"Best CV score: {grid_search.best_score_:.4f}")


# ======================================================================
# 9. Evaluate on train + test
# ======================================================================
y_train_pred    =    best_clf.predict(X_train)    y_test_pred        =
best_clf.predict(X_test)
```

```
  train_acc       =       accuracy_score(y_train,
y_train_pred) test_acc   = accuracy_score(y_test,
y_test_pred)

metrics_table = pd.DataFrame({
```

```
    "metric": ["train accuracy", "test accuracy", "overfit gap"],
    "value":            [round(train_acc,      4),
round(test_acc, 4),
              round(train_acc - test_acc, 4)],
})
show_table("Model Accuracy", metrics_table)


# Full classification report (test)
report_dict = classification_report(y_test, y_test_pred,
                                    output_dict=True, zero_division=0)
report_df = pd.DataFrame(report_dict).T.reset_index() report_df =
report_df.rename(columns={"index": "class"})
report_df = report_df.round(3)
show_table("Classification Report  Test Set", report_df)
# ======================================================================
# 10. Graph: Confusion matrix heatmap
# ====================================================================== labels
= sorted(y_test.unique())
cm = confusion_matrix(y_test, y_test_pred, labels=labels)


fig, ax = plt.subplots(figsize=(12, 10)) sns.heatmap(cm,
annot=True, fmt="d", cmap="Blues",
xticklabels=labels, yticklabels=labels,
cbar_kws={"label": "count"}, ax=ax)
ax.set_title("Confusion Matrix  Test Set", fontsize=14, fontweight="bold")
ax.set_xlabel("Predicted") ax.set_ylabel("Actual")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0) plt.tight_layout()
cm_path = os.path.join(OUTPUT_DIR, "confusion_matrix.png")
plt.savefig(cm_path, bbox_inches="tight") plt.close()
print(f"\n[Chart saved] {cm_path}")


# ======================================================================
# 11. Graph: Per-class F1 scores
# ====================================================================== per_class
= report_df[report_df["class"].isin(labels)][["class", "precision",
"recall", "f1-score"]] per_class_melt = per_class.melt(id_vars="class",
                               var_name="metric", value_name="score")


fig, ax = plt.subplots(figsize=(13, 8))
sns.barplot(data=per_class_melt, x="score", y="class",
hue="metric", palette="Set2", ax=ax)
ax.set_title("Precision / Recall / F1 per Category (Test
Set)",              fontsize=14, fontweight="bold")
ax.set_xlim(0, 1.05) ax.set_xlabel("Score") ax.set_ylabel("")
ax.legend(loc="lower right") plt.tight_layout()
f1_path = os.path.join(OUTPUT_DIR, "per_class_scores.png")
plt.savefig(f1_path, bbox_inches="tight") plt.close()
print(f"[Chart saved] {f1_path}")


# ======================================================================
# 12. Graph: Feature importance (top-N numeric + top-N merchant n-grams)
# ======================================================================
try:     classifier = best_clf.named_steps["classifier"]     pre =
best_clf.named_steps["preprocessor"]


    # Names from the ColumnTransformer
    tfidf_names = pre.named_transformers_["merchant_tfidf"].get_feature_names_out()
num_names   = numeric_cols
    feature_names = np.concatenate([tfidf_names, num_names])


    importances = classifier.feature_importances_
top_n = 25
    idx = np.argsort(importances)[-top_n:]


    imp_df = pd.DataFrame({
        "feature": feature_names[idx],
        "importance": importances[idx],
    })
    show_table(f"Top {top_n} Most Important Features",
```

```
                imp_df.sort_values("importance", ascending=False))


    fig, ax = plt.subplots(figsize=(11, 8))
    sns.barplot(data=imp_df, x="importance", y="feature",
palette="crest", ax=ax)
    ax.set_title(f"Top {top_n} Feature Importances", fontsize=14,
fontweight="bold")     ax.set_xlabel("Importance")     ax.set_ylabel("")
plt.tight_layout()
    fi_path = os.path.join(OUTPUT_DIR,
"feature_importance.png")     plt.savefig(fi_path,
bbox_inches="tight")     plt.close()
    print(f"[Chart saved] {fi_path}") except
Exception as e:     print(f"(Feature-importance chart
skipped: {e})")


# ======================================================================
# 13. Cross-validation
# ====================================================================== print("\n"
+ "=" * 70)
print("Running 5-fold cross-validation on full dataset...")
cv_scores = cross_val_score(best_clf, df.drop(columns=["category", "date"]),
df["category"], cv=5)


cv_table = pd.DataFrame({
"fold":     [1, 2, 3, 4, 5],
    "accuracy": np.round(cv_scores, 4),
})
show_table("5-Fold Cross-Validation", cv_table)
print(f"Mean CV accuracy: {cv_scores.mean():.4f}  (± {cv_scores.std():.4f})")


# CV scores plot
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(x=[f"Fold {i+1}" for i in
range(len(cv_scores))],             y=cv_scores,
palette="viridis", ax=ax) ax.axhline(cv_scores.mean(),
color="red", linestyle="--",            label=f"Mean =
{cv_scores.mean():.4f}") ax.set_ylim(0, 1.05)
ax.set_ylabel("Accuracy")
ax.set_title("Cross-Validation Accuracy per Fold",
fontsize=13, fontweight="bold") ax.legend()
plt.tight_layout()
cv_path = os.path.join(OUTPUT_DIR, "cv_scores.png")
plt.savefig(cv_path, bbox_inches="tight") plt.close()
print(f"[Chart saved] {cv_path}")


# ====================================================================== #
14. Save model
#
======================================================================
joblib.dump(best_clf, MODEL_PATH) print(f"\n{'=' * 70}")
print(f"Model saved at: {MODEL_PATH}") print(f"All
charts saved under: {OUTPUT_DIR}") print("=" * 70)
```

Test Model Codes

test_model.py loads the saved pipeline and the Testing.csv hold-out set, predicts every transaction, and reports accuracy, weighted precision, recall and F1-score, the per-class classification report, confusion matrices (counts and normalised), actual versus predicted counts, prediction confidence and the list of misclassified transactions. All tables are printed and all charts and CSV files are saved to the outputs folder.

```
"""
test_model.py
Evaluates the trained AI Budget Assistant model on Testing.csv. Prints
rich tables and saves charts to outputs/.
"""   import
os import
warnings
warnings.filterwarnings("ignore")
  import numpy as
np import pandas
as pd import
joblib
  import matplotlib
matplotlib.use("Agg") import
matplotlib.pyplot as plt
import seaborn as sns


from sklearn.metrics import (
    classification_report, confusion_matrix, accuracy_score,
    precision_score, recall_score, f1_score,
)
from tabulate import tabulate


from config import TEST_CSV_PATH, MODEL_PATH, OUTPUT_DIR
  sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 110



# ====================================================================
# Helper: pretty table
# ==================================================================== def
show_table(title, df):
    print(f"\n=== {title} ===")
    print(tabulate(df, headers="keys", tablefmt="fancy_grid",
                   floatfmt=",.3f", showindex=False))



# ====================================================================
# 1. Load model + test data
# ==================================================================== print("="
* 70)
print("AI BUDGET ASSISTANT  MODEL TESTING") print("="
* 70)


print(f"\nLoading model  : {MODEL_PATH}")
model = joblib.load(MODEL_PATH)


print(f"Loading test set: {TEST_CSV_PATH}") df
= pd.read_csv(TEST_CSV_PATH)
  if "date" in df.columns:     df
= df.drop(columns=["date"])


X_test = df.drop(columns=["category"]) y_true
= df["category"]


show_table("Test Set Snapshot (first 10 rows)",
df.head(10))


# ====================================================================
# 2. Predict
#
====================================================================
print("\nRunning predictions on test set...") y_pred =
model.predict(X_test)
# ====================================================================
```

```
# 3. Global metrics
# ========================================================================== acc
= accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred, average="weighted", zero_division=0)
rec  = recall_score(y_true, y_pred, average="weighted", zero_division=0) f1
= f1_score(y_true, y_pred, average="weighted", zero_division=0)


metrics_table = pd.DataFrame({
    "metric": ["Accuracy", "Precision (weighted)",
               "Recall (weighted)", "F1-score (weighted)"],
    "value":  [round(acc, 4), round(prec, 4),
round(rec, 4), round(f1, 4)],
})
show_table("Overall Test Performance", metrics_table)


# ========================================================================
# 4. Classification report as a table
# ========================================================================== report_dict
= classification_report(
    y_true, y_pred, output_dict=True, zero_division=0
)
report_df = pd.DataFrame(report_dict).T.reset_index()
report_df = report_df.rename(columns={"index": "class"}).round(3)


show_table("Classification Report (per class)", report_df)
# Also keep the plain-text version for the saved file
report_text = classification_report(y_true, y_pred, zero_division=0)
# ========================================================================
# 5. Confusion matrix  table + heatmap
# ========================================================================== labels
= sorted(y_true.unique())
cm = confusion_matrix(y_true, y_pred, labels=labels) cm_df
= pd.DataFrame(cm, index=labels, columns=labels)
  show_table("Confusion Matrix (counts)", cm_df.reset_index()
.rename(columns={"index": "actual\\predicted"}))
# Normalised matrix (row = actual, values sum to 1) cm_norm
= pd.DataFrame(
    cm.astype(float) / cm.sum(axis=1, keepdims=True),
index=labels, columns=labels
).round(3)


show_table("Confusion Matrix (normalised)",
           cm_norm.reset_index().rename(columns={"index": "actual\\predicted"}))


# Heatmap (counts)
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
xticklabels=labels, yticklabels=labels,
cbar_kws={"label": "count"}, ax=ax)
ax.set_title("Confusion Matrix  Test Set (counts)",
fontsize=14, fontweight="bold")
ax.set_xlabel("Predicted") ax.set_ylabel("Actual")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0) plt.tight_layout()
cm_png = os.path.join(OUTPUT_DIR,
"test_confusion_matrix.png") plt.savefig(cm_png,
bbox_inches="tight") plt.close()
print(f"\n[Chart saved] {cm_png}")


# Heatmap (normalised)
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(cm_norm, annot=True, fmt=".2f",
cmap="Greens",             xticklabels=labels,
yticklabels=labels,             cbar_kws={"label":
"proportion"}, ax=ax) ax.set_title("Confusion Matrix
Test Set (normalised)",              fontsize=14,
fontweight="bold") ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0) plt.tight_layout()
cm_norm_png = os.path.join(OUTPUT_DIR,
"test_confusion_matrix_normalised.png") plt.savefig(cm_norm_png,
bbox_inches="tight") plt.close()
```

```
print(f"[Chart saved] {cm_norm_png}")
```

```
# ======================================================================
# 6. Per-class bar chart (precision / recall / f1)
#
======================================================================
per_class = report_df[report_df["class"].isin(labels)][     ["class",
"precision", "recall", "f1-score"]
]
per_class_melt = per_class.melt(
    id_vars="class", var_name="metric", value_name="score"
)
fig, ax = plt.subplots(figsize=(13, max(6, 0.45 * len(labels))))
sns.barplot(data=per_class_melt, x="score", y="class",
hue="metric", palette="Set2", ax=ax)
ax.set_title("Precision / Recall / F1 per Category (Test
Set)",              fontsize=14, fontweight="bold")
ax.set_xlim(0, 1.05) ax.set_xlabel("Score") ax.set_ylabel("")
ax.legend(loc="lower right") plt.tight_layout()
per_class_png = os.path.join(OUTPUT_DIR,
"test_per_class_scores.png") plt.savefig(per_class_png,
bbox_inches="tight") plt.close()
print(f"[Chart saved] {per_class_png}")


# ======================================================================
# 7. Actual vs Predicted distribution
# ====================================================================== actual_counts
= y_true.value_counts().reindex(labels).fillna(0)
predicted_counts = pd.Series(y_pred).value_counts().reindex(labels).fillna(0)
  compare_df =
pd.DataFrame({
    "category":  labels,
    "actual":    actual_counts.values.astype(int),
    "predicted": predicted_counts.values.astype(int),
    "diff":      (predicted_counts.values - actual_counts.values).astype(int),
})
show_table("Actual vs Predicted Counts", compare_df)


comp_long = compare_df.melt(id_vars="category",
                            value_vars=["actual", "predicted"],
var_name="type", value_name="count")


fig, ax = plt.subplots(figsize=(13, max(6, 0.45 * len(labels))))
sns.barplot(data=comp_long, y="category", x="count",
hue="type", palette="coolwarm", ax=ax)
ax.set_title("Actual vs Predicted Category Counts",
fontsize=14, fontweight="bold")
ax.set_xlabel("Number of transactions")
ax.set_ylabel("") plt.tight_layout()
counts_png = os.path.join(OUTPUT_DIR,
"test_actual_vs_predicted.png") plt.savefig(counts_png,
bbox_inches="tight") plt.close()
print(f"[Chart saved] {counts_png}")


# ======================================================================
# 8. Confidence of predictions
#
======================================================================
try:     if hasattr(model, "predict_proba"):         proba =
model.predict_proba(X_test)         confidence = proba.max(axis=1)


        conf_df = pd.DataFrame({
            "stat":  ["min", "25%", "median", "mean", "75%", "max"],
            "confidence": [
                round(float(np.min(confidence)), 4),
                round(float(np.percentile(confidence, 25)), 4),
                round(float(np.median(confidence)), 4),
round(float(np.mean(confidence)), 4),
round(float(np.percentile(confidence, 75)), 4),
round(float(np.max(confidence)), 4),
            ],
})
        show_table("Prediction Confidence (max probability)", conf_df)
          fig, ax = plt.subplots(figsize=(9, 5))
sns.histplot(confidence, bins=20, kde=True,
```

```
                    color="#4C72B0", ax=ax)
        ax.axvline(float(np.mean(confidence)),
color="red", linestyle="--",
                   label=f"Mean =
{np.mean(confidence):.3f}")
ax.set_title("Distribution of Prediction Confidence",
fontsize=14, fontweight="bold")         ax.set_xlabel("Max
predicted probability")         ax.set_ylabel("Number of
test samples")         ax.legend()
plt.tight_layout()
        conf_png = os.path.join(OUTPUT_DIR,
"test_confidence_distribution.png")         plt.savefig(conf_png,
bbox_inches="tight")         plt.close()
        print(f"[Chart saved] {conf_png}")
except Exception as e:
print(f"(Confidence chart skipped: {e})")


# ======================================================================
# 9. Errors breakdown  which categories get confused the most
#
======================================================================
wrong = y_true != y_pred error_df = pd.DataFrame({
    "merchant":  X_test["merchant"].astype(str).values[wrong],
    "amount":    X_test["amount"].values[wrong],
    "actual":    y_true.values[wrong],
    "predicted": np.asarray(y_pred)[wrong],
})
error_df = error_df.sort_values("actual").reset_index(drop=True)


show_table("Misclassified Transactions (sample of up to 30)",
error_df.head(30))


err_path = os.path.join(OUTPUT_DIR,
"test_misclassified.csv") error_df.to_csv(err_path,
index=False) print(f"\nFull error list saved: {err_path}")


# ======================================================================
# 10. Save summary files
# ======================================================================
report_path = os.path.join(OUTPUT_DIR, "classification_report.txt") cm_path
= os.path.join(OUTPUT_DIR, "confusion_matrix.csv")
cm_norm_path= os.path.join(OUTPUT_DIR, "confusion_matrix_normalised.csv")
metrics_path= os.path.join(OUTPUT_DIR, "test_metrics.csv")
  with open(report_path, "w") as
f:
    f.write(f"Accuracy : {acc:.4f}\n")
    f.write(f"Precision: {prec:.4f}\n")
    f.write(f"Recall   : {rec:.4f}\n")
    f.write(f"F1       : {f1:.4f}\n\n")
    f.write(report_text)
cm_df.to_csv(cm_path) cm_norm.to_csv(cm_norm_path)
metrics_table.to_csv(metrics_path, index=False)
  print(f"\n{'=' * 70}")
print("SUMMARY OF
OUTPUTS") print("=" * 70)
for label, p in [
    ("Report            ", report_path),
    ("Confusion matrix  ", cm_path),
    ("Confusion (norm)  ", cm_norm_path),
    ("Metrics           ", metrics_path),
    ("Misclassified     ", err_path),
    ("CM chart          ", cm_png),
    ("CM chart (norm)   ", cm_norm_png),
    ("Per-class scores  ", per_class_png),
("Actual vs predict ", counts_png),
]:       print(f"   {label}
{p}") print("=" * 70)
```

Predict Codes

predict.py loads the trained pipeline and categorises a single transaction (interactive mode) or a whole CSV file (batch mode). The budget logic then builds on these predictions: make_budget.py derives the monthly budget from historical spending, and the analyse() function of budget_assistant.py totals the predicted spending per

category and raises the 80% warnings and 100% alerts. The same functions are used by the Streamlit app (app.py). **predict.py**

```
import os import
sys import pandas
as pd import
joblib


from config import MODEL_PATH, OUTPUT_DIR


model = joblib.load(MODEL_PATH)


FEATURES = ["merchant", "amount",
            "day_of_month", "day_of_week", "is_weekend", "is_month_end"]


def predict_one(merchant, amount,
                day_of_month=15, day_of_week=2,
is_weekend=0, is_month_end=0):     row = {
        "merchant": merchant, "amount": amount,
        "day_of_month": day_of_month, "day_of_week": day_of_week,
"is_weekend": is_weekend, "is_month_end": is_month_end,
    }
    return model.predict(pd.DataFrame([row]))[0]
def predict_batch(df):     for col, default in [
        ("day_of_month", 15), ("day_of_week", 2),
("is_weekend", 0), ("is_month_end", 0),
    ]:         if col not in
df.columns:
            df[col] = default
    df["predicted_category"] = model.predict(df[FEATURES])
return df
    def
interactive():
    print("\n=== PREDICT MODE ===")
print("Type 'done' when finished.\n")     while
True:         m = input("Merchant : ").strip()
if m.lower() == "done":             break
try:             a = float(input("Amount   :
").strip())         except ValueError:
print("Invalid amount.\n")             continue
        print(f"  -> Predicted category: {predict_one(m, a)}\n")
    def csv_batch(path):
df = pd.read_csv(path)
    df = df.loc[:, ~df.columns.str.contains("^Unnamed")]
df.columns = [c.strip() for c in df.columns]
      df = df.rename(columns={
"Description": "merchant",
        "Amount": "amount",
    })
    out = predict_batch(df)
    out_path = os.path.join(OUTPUT_DIR, "predictions.csv")
out.to_csv(out_path, index=False)     print(f"Predictions
saved: {out_path}")
print(out.head(20).to_string(index=False))
```

```
  if __name__ ==
"__main__":     if
len(sys.argv) > 1:
        csv_batch(sys.argv[1])
else:         interactive()
```

make_budget.py budget derivation (excerpt)

```
BUFFER = 1.10     # 10 % buffer over average monthly spend
# ... (data loading and monthly aggregation omitted) ...


avg   = monthly.groupby("category")["amount"].mean().round(2)


budget = (avg * BUFFER).round(0).astype(int).reset_index() budget.columns
= ["category", "monthly_budget"]
budget = budget.sort_values("category").reset_index(drop=True)
```

budget_assistant.py analyse() with the 80% and 100% alert logic (excerpt)

```
def analyse(transactions):     totals =
{cat: 0.0 for cat in BUDGET}     details
= []
      for t in
transactions:         cat
= predict_one(
            t["merchant"], float(t["amount"]),
            t.get("day_of_month", 15), t.get("day_of_week", 2),
            t.get("is_weekend", 0), t.get("is_month_end", 0),
)
        totals[cat] = totals.get(cat, 0.0) + float(t["amount"])
details.append({**t, "category": cat})
      alerts = []     for cat, spent
in totals.items():         budget =
BUDGET.get(cat, 0)         if budget
<= 0:             continue
ratio = spent / budget
remaining = budget - spent
if ratio >= DANGER_THRESHOLD:
alerts.append({
"level": "OVER",
                "category": cat,
                "spent": spent,
                "budget": budget,
                "remaining": remaining,
                "pct": ratio * 100,
                "message": f"OVER BUDGET  - {cat}: spent {money(spent)} of "
f"{money(budget)} (over by {money(abs(remaining))})",
            })         elif ratio >=
WARN_THRESHOLD:
alerts.append({
"level": "WARN",
                "category": cat,
                "spent": spent,
                "budget": budget,
                "remaining": remaining,
                "pct": ratio * 100,
                "message": f"APPROACHING - {cat}: spent {money(spent)} of "
f"{money(budget)} (remaining {money(remaining)})",             })

    return {"details": details, "totals": totals, "alerts": alerts}
```

Example predictions from the trained model

Merchant	Amount	Predicted category	Confidence
Thai Restaurant	24.22	Restaurants	98%
Netflix	11.76	Movies & DVDs	58%
Gas Company	45.00	Utilities	100%

Predictions use default date features (day 15, a Wednesday, not a weekend or month-end), as in predict.py and app.py.

Grammarly Report

Report: (AI Budget Assistant)_Documentation | by _____ **General Metrics**

Characters	Words	Sentences	Reading time	Speaking time
13,842	1,912	258	7 min 32 sec	14 min 41 sec

84% This text scores better than 84% of all texts checked by Grammarly.

Writing Issues

Correctness: 1 determiner use (a/an/the/this, etc.), 1 misuse of modifiers, 1 incomplete sentence
Clarity: wordy sentences

Vocabulary

37% rare words
Average word length: 5.8 characters per word
Average sentence length: 7.2 words per sentence

Report was generated on Monday, Oct 13, 2025, 12:10 PM

References

Tutorialspoint (2024). *Machine Learning Tutorial – Tutorialspoint*. [online] www.tutorialspoint.com. Available at: https://www.tutorialspoint.com/machine-learning/index.html

www.tutorialspoint.com. (n.d.). *AI with Python Tutorial* [online] Available at: https://www.tutorialspoint.com/artificial_intelligence_with_python/index.html Matsela, M. (n.d.). *Lesson 3 – AI with Python – Part 1*.

Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research.

GitHub Repository https://github.com/AI-Python-Projects/AI-Budget-Assistance.git

End of Documentation